Estudio Comparativo de Métricas en la segmentación de imágenes mediante Spectral Clustering

26 de marzo de 2026

1. Introducción

En esta parte se analiza el comportamiento del algoritmo de Clustering Espectral ante imágenes con iluminación no uniforme. A menudo, las variaciones drásticas de luz (como sombras proyectadas o brillos especulares) inducen a los algoritmos de segmentación a fragmentar los objetos de forma errónea. Para demostrar este fenómeno, se evalúa el rendimiento de dos métricas espaciales (Euclídea y Coseno) sobre una imagen sintética denominada "Esfera de Neón", la cual presenta un fuerte degradado radial de color verde sobre un fondo azul oscuro liso.

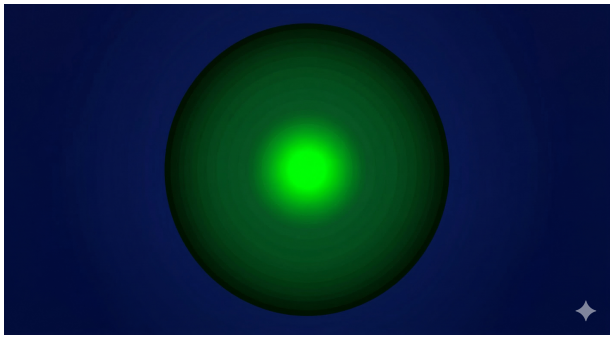

Figura 1: Imagen original: la "Esfera de Neón".

2. Fundamento Teórico de la Distancia Euclídea

2.1. El Origen: El Teorema de Pitágoras

La distancia Euclídea representa la métrica más intuitiva para medir el espacio, calculando la distancia geométrica en línea recta entre dos puntos. Su formulación matemática se deriva directamente del **Teorema de Pitágoras** ($c^2 = a^2 + b^2$). Para dos puntos $P = (p_1, p_2)$ y $Q = (q_1, q_2)$ en un plano bidimensional, la distancia se define como:

$$d(P, Q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2} \tag{1}$$

2.2. Generalización a N Dimensiones (Espacio Euclídeo)

En el ámbito de la Visión Artificial y la Ciencia de Datos, los elementos suelen representarse como vectores en un espacio de n dimensiones ($\mathbb{R}^n$). La base pitagórica se

generaliza sumando las diferencias al cuadrado de cada componente. Para dos vectores $X = [x_1, x_2, \ldots, x_n]$ e $Y = [y_1, y_2, \ldots, y_n]$, la distancia Euclídea (también conocida como Norma L_2) se expresa como:

$$d(X, Y) = ||X - Y||_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{2}$$

2.3. Aplicación al Espacio de Color RGB

En la segmentación de imágenes, cada píxel representa un punto en un **cubo tridimensional de color**, definido por los ejes Rojo (R), Verde (G) y Azul (B), con valores en el rango discreto de $[0, 255]$. La distancia computacional entre dos píxeles X e Y se calcula mediante:

$$d_{euclidea}(X, Y) = \sqrt{(R_X - R_Y)^2 + (G_X - G_Y)^2 + (B_X - B_Y)^2} \tag{3}$$

2.4. Comportamiento Matemático: La Sensibilidad a la Magnitud

Cualquier vector de color posee dos propiedades algebraicas fundamentales: la **dirección** (que define el matiz o cromaticidad) y la **magnitud** (su longitud desde el origen, que determina la intensidad lumínica).

La distancia Euclídea es extremadamente sensible a las variaciones de magnitud. Si la iluminación disminuye, los valores del vector se aproximan al origen $[0, 0, 0]$. La métrica interpreta este descenso de intensidad como un desplazamiento gigantesco a través del espacio de color, ignorando por completo que la dirección (el color base) permanece intacta.

Analizando empíricamente los puntos clave de la imagen de prueba (Esfera de Neón):

- **Vector A (Centro Brillante):** $[0, 255, 0]$. Su magnitud (longitud) es 255.
- **Vector B (Borde Oscuro):** $[0, 20, 0]$. Su magnitud (longitud) es 20.
- **Vector C (Fondo Azul):** $[0, 0, 50]$. Su magnitud (longitud) es 50.

Al calcular las distancias analíticas componente a componente:

$$d(B, A) = \sqrt{(0-0)^2 + (20-255)^2 + (0-0)^2} = \sqrt{(-235)^2} = 235$$
$$d(B, C) = \sqrt{(0-0)^2 + (20-0)^2 + (0-50)^2} = \sqrt{400 + 2500} \approx 53{,}85$$

2.5. Conclusión Teórica

Matemáticamente, la métrica Euclídea construye hiper-esferas de similitud alrededor de cada punto, agrupando aquellos elementos que caen dentro de un radio numérico específico. Puesto que el verde oscuro de la esfera (B) se encuentra espacialmente dentro de la región de similitud geométrica del azul oscuro del fondo (C) —están a una distancia de 53.85—, el algoritmo los agrupa en el mismo clúster.

El sistema carece de mecanismos algebraicos para identificar que el vector B es simplemente una proyección acortada (misma dirección, distinta magnitud) del vector A. Por consiguiente, en técnicas de *Clustering Espectral* aplicadas sobre imágenes reales con sombras proyectadas, superficies curvas o luz direccional, la métrica Euclídea es incapaz de aislar correctamente las topologías de los objetos, ya que castiga drásticamente la atenuación de la luz por encima del cambio cromático.

3. Fundamento Teórico de la Similitud del Coseno

3.1. El Origen: Producto Escalar y Trigonometría

A diferencia de las métricas espaciales tradicionales que evalúan la separación física entre las coordenadas finales de dos vectores, la similitud del Coseno mide el **ángulo (θ)** que se forma entre ellos cuando ambos se proyectan desde el mismo punto de origen.

Matemáticamente, este concepto se deriva de la definición geométrica del **producto escalar** (o producto punto) de dos vectores X e Y:

$$X \cdot Y = ||X|| \times ||Y|| \times \cos(\theta) \tag{4}$$

Despejando el término trigonométrico, obtenemos la formulación fundamental de la Similitud del Coseno:

$$\cos(\theta) = \frac{X \cdot Y}{||X|| \times ||Y||} \tag{5}$$

Para implementar este concepto como una métrica de disimilitud o distancia (donde 0,0 indica identidad absoluta y 1,0 disimilitud máxima en el cuadrante positivo), la **Distancia Coseno** se define como:

$$d_{coseno}(X, Y) = 1 - \cos(\theta) = 1 - \frac{X \cdot Y}{||X|| \times ||Y||} \tag{6}$$

3.2. Generalización a N Dimensiones

En un espacio analítico de n dimensiones ($\mathbb{R}^n$), el producto escalar equivale a la suma del producto componente a componente, y la magnitud ($||X||$) corresponde a la raíz cuadrada de la suma de sus componentes al cuadrado. Por tanto, la expresión computacional exacta es:

$$d_{coseno}(X, Y) = 1 - \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \sqrt{\sum_{i=1}^{n} y_i^2}} \tag{7}$$

3.3. Aplicación al Espacio de Color RGB e Invarianza a la Magnitud

En el procesamiento de imágenes, cada píxel es tratado como un vector $V = [R, G, B]$. Al aplicar la métrica Coseno, el algoritmo evalúa exclusivamente la **dirección cromática** en el espacio 3D, ignorando por completo la longitud del vector.

Esta propiedad, conocida como **invarianza a la magnitud**, normaliza los datos al dividir el producto escalar por las magnitudes de los vectores ($||X|| \times ||Y||$). Esto cancela el efecto de la luminancia (brillo) y aísla la proporción de color puro (cromaticidad).

Para demostrarlo algebraicamente, se evalúan los mismos tres puntos de la Esfera de Neón:

- **Vector A (Centro Brillante):** $A = [0, 255, 0]$, con magnitud $||A|| = 255$.
- **Vector B (Borde Oscuro):** $B = [0, 20, 0]$, con magnitud $||B|| = 20$.
- **Vector C (Fondo Azul):** $C = [0, 0, 50]$, con magnitud $||C|| = 50$.

Caso 1: Verde brillante vs. Verde oscuro (Cambio de intensidad, mismo matiz)

$$A \cdot B = (0 \times 0) + (255 \times 20) + (0 \times 0) = 5100$$
$$\cos(A, B) = \frac{5100}{255 \times 20} = \frac{5100}{5100} = 1$$

Dado que $\cos(0°) = 1$, la distancia es $d = 1 - 1 = \mathbf{0{,}0}$. Los vectores son colineales; el algoritmo los reconoce como cromáticamente idénticos.

Caso 2: Verde oscuro vs. Fondo Azul (Misma penumbra, distinto matiz)

$$B \cdot C = (0 \times 0) + (20 \times 0) + (0 \times 50) = 0$$
$$\cos(B, C) = \frac{0}{20 \times 50} = \frac{0}{1000} = 0$$

Dado que $\cos(90°) = 0$, la distancia es $d = 1 - 0 = \mathbf{1{,}0}$. Los vectores son ortogonales, arrojando la máxima separación angular posible en el espacio RGB.

3.4. Conclusión Teórica

A diferencia de la distancia Euclídea, que traza hiper-esferas rígidas, la distancia Coseno proyecta **conos de similitud que parten desde el origen**. Cualquier vector que yazca sobre el mismo rayo direccional se considera perteneciente a la misma clase, sin importar si su magnitud lo sitúa en una zona de alta iluminación (lejos del origen) o en una zona de penumbra profunda (cerca del origen).

En consecuencia, la métrica Coseno se presenta como una herramienta topológica inmensamente superior para la segmentación de objetos que presentan volúmenes curvos, gradientes radiales o sombras arrojadas, ya que descarta los sesgos de luminancia y agrupa las regiones basándose estrictamente en su firma cromática.

4. Análisis de la implementación en Jupyter

Para realizar un test entre ambas métricas, fue necesario adpatar la matriz de características del algoritmo original:

- Eliminación de X, Y: el código original concatenaba la posición espacial del píxel con su color. Para la métrica Coseno, calcular el ángulo de un vector que mezcla espacio y color carece de sentido topológico. Por tanto, se construyó una matriz exclusivamente basada en los canales RGB.

- División por 0: la métrica Coseno exige dividir entre la magnitud del vector. Para evitar que los píxeles en negro absoluto ([0,0,0]) generaran un error computacional, se añadió una constante muy pequeña, 10^{-5} a todos los canales.

4.1. Análisis de la Métrica Euclídea: El Fracaso Morfológico

```python
# Construccion de la Matriz Final (Ignorando espacio fisico)
X_features_trampa = np.column_stack((R, G, B))

# Calculo de la Matriz de Distancias (Euclidea)
```

```python
dist_matrix = cdist(X_features_trampa, X_features_trampa, metric='euclidean')

# Matriz de Afinidad Gaussiana
matriz_afinidad = calcular_mi_afinidad(dist_matrix, tipo='gaussiana', parametro=0.1)
```

Listing 1: Cálculo de afinidad con métrica Euclídea

En la celda 4, se calcula la matriz de distancias usando la métrica euclidea. Al calcular esta distancia sobre mi esfera, el algortimo percibe saltos numéricos enormes en la intensidad de la luz (desde el centro brillantes hasta los bordes oscuros). La función Gaussiana es tan estricta que asigna una conexión de cero a la relación entre el centro brillante y el borde oscuro. Se interpreta que la imagen se rompen en las siguientes 5 partes:

1. El núcleo central.
2. Un anillo intermedio claro.
3. Un anillo intermedio oscuro.
4. El borde exteior.
5. El fondo azul.

```python
# Calculo de Matriz Laplaciana Normalizada
L_norm = csgraph.laplacian(W_matriz, normed=True)
autovalores, _ = eigsh(L_norm, k=10, which='SA')
autovalores = np.sort(autovalores)

# Busqueda del salto maximo (Eigengap)
gaps = np.diff(autovalores)
k_optimo = np.argmax(gaps[1:]) + 2 # El resultado arroja k=5
```

Listing 2: Heurística del Eigengap evidenciando la sobresegmentación

En la celda 5, se calcula la matriz Laplaciana y se extraen sus autovalores. La métrica euclídea ha destrozado el gradiente continuo de la esfera y el Laplaciano detecta 5 bloques aislados y devuelve que el salto óptimo está en $k = 5$.

Este resultado de $k = 5$ es la prueba computacional del fenómeno de **sobresegmentación**. La métrica Euclídea ha destrozado el gradiente continuo de la esfera, interpretando los diferentes escalones de intensidad lumínica como si fueran cinco anillos concéntricos totalmente desconectados.

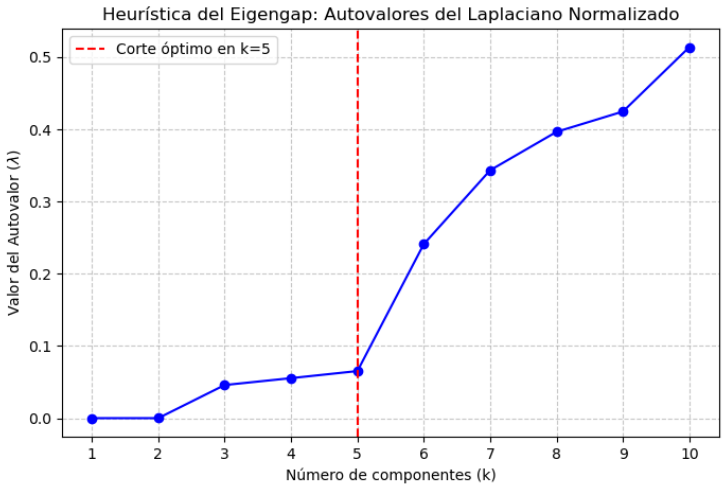

Figura 2: Gráfica de autovalores mostrando el corte óptimo en $k = 5$.

Ejecutando el último paso del algoritmo en el que se llama a la fucnión `SpecttralClustering`, se obtiene el siguiente resultado:

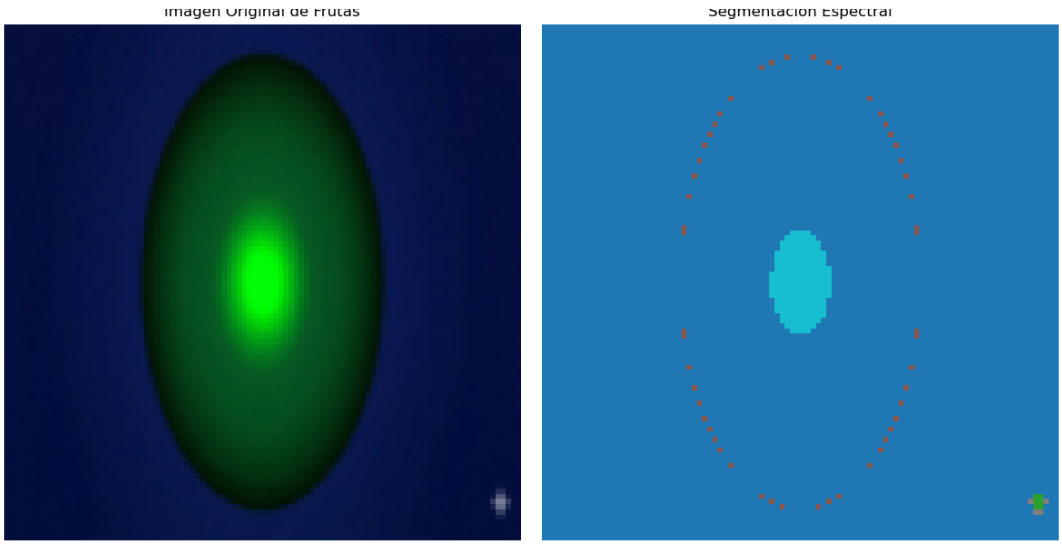

Figura 3: Fracaso de la segmentación Euclídea: el algoritmo deforma el objeto central y agrupa sus bordes con el fondo.

5. Análisis de la Implementación: El Éxito de la Métrica Coseno

Para contrarrestar la sobresegmentación producida por la métrica Euclídea, se implementó una variante del algoritmo basada estrictamente en la **Similitud del Coseno**. El objetivo es demostrar cómo la invarianza a la magnitud permite aislar correctamente la morfología del objeto, dejando que sea la propia matemática espectral la que deduzca el número de elementos en la imagen.

5.1. Adaptación de la Matriz de Características

Para que la métrica Coseno opere de forma topológicamente coherente, se aislaron los canales cromáticos (eliminando las coordenadas espaciales X, Y). Además, dado que la métrica Coseno divide el producto escalar entre las magnitudes de los vectores, se añadió una constante de estabilidad numérica (10^{-5}) a todos los canales para evitar que los píxeles en oscuridad absoluta (vector nulo $[0, 0, 0]$) indujeran un error de división por cero.

```
# Extraccion de canales RGB aplanados con "Seguro Matematico" (1e-5)
R = img_rgb[:, :, 0].flatten() + 1e-5
G = img_rgb[:, :, 1].flatten() + 1e-5
B = img_rgb[:, :, 2].flatten() + 1e-5

# Construccion de la Matriz Final puramente cromatica
X_features_trampa = np.column_stack((R, G, B))

# 1. Calculo de la Matriz de Distancias (Coseno)
dist_matrix_cos = cdist(X_features_trampa, X_features_trampa, metric='
    cosine')

# 2. Matriz de Afinidad Gaussiana adaptada al rango angular
matriz_afinidad_cos = calcular_mi_afinidad(dist_matrix_cos, tipo='
    gaussiana', parametro=0.05)
```

Listing 3: Preparación de la matriz RGB con factor de estabilidad para la métrica Coseno

5.2. El Eigengap y la Solidez Topológica

A diferencia de la métrica Euclídea (que fracturaba el objeto arrojando un $k = 5$), la métrica Coseno comprende que todo el degradado verde comparte la misma dirección cromática. Al evaluar los autovalores del Laplaciano construido con esta matriz de afinidad, la heurística del Eigengap revela naturalmente un salto máximo en $k = 2$. El algoritmo detecta de forma automática y perfecta la existencia de dos únicos clústeres independientes (Objeto y Fondo).

```
# Matriz Laplaciana Normalizada para el Coseno
W_matriz_cos = csr_matrix(matriz_afinidad_cos)
L_norm_cos = csgraph.laplacian(W_matriz_cos, normed=True)

# Calculo de los 10 primeros autovalores
autovalores_cos, _ = eigsh(L_norm_cos, k=10, which='SA')
autovalores_cos = np.sort(autovalores_cos)

# Busqueda automatica del salto maximo (Eigengap)
gaps_cos = np.diff(autovalores_cos)
k_optimo_cos = np.argmax(gaps_cos[1:]) + 2 # El resultado arroja
    automaticamente k=2
```

Listing 4: Cálculo dinámico del número de clústeres mediante Autovalores

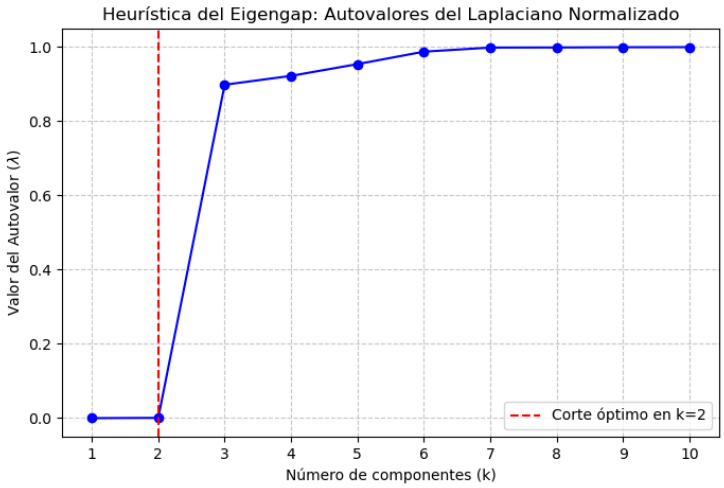

Figura 4: Heurística del Eigengap con métrica Coseno mostrando una separación perfecta y calculando $k = 2$.

5.3. Resultado de la Segmentación Automática

Con el parámetro k deducido matemáticamente, se ejecuta el modelo de segmentación. Al basarse puramente en la dirección del color (matiz) y descartar la longitud del vector (brillo), la métrica logra vincular todo el degradado verde como un único clúster topológicamente sólido.

```
# Ejecucion del modelo usando el k=2 obtenido por el Eigengap
sc_cos = SpectralClustering(n_clusters=k_optimo_cos, affinity='
    precomputed', random_state=42)
etiquetas_cos = sc_cos.fit_predict(matriz_afinidad_cos)
```

Listing 5: Ejecución del Clustering Espectral con el k óptimo calculado

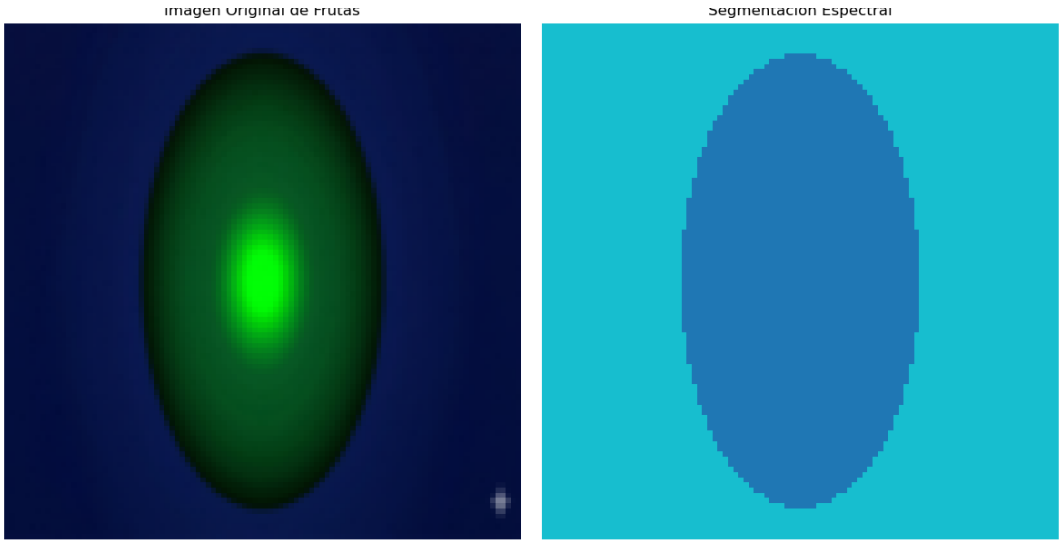

Figura 5: Éxito de la segmentación Coseno.

Como se observa en la Figura 5, el algoritmo extrae el círculo perfecto conservando su morfología, radio y dimensiones reales, demostrando total inmunidad frente a la trampa óptica generada por el gradiente radial de sombra.